GenAI Field Notes: Federal Funds Matchmaker (finals)

Team 14 · UNC Solve-a-thon 2026 · NC DHHS

1. Tools we used

Tool	Where	What for
Azure OpenAI **text-embedding-3-small** (512 dimensions)	In the product, daily job	Meaning-based similarity between each opportunity and the 20 plan objectives
Azure OpenAI **gpt-4.1-mini** (JSON mode, temperature 0)	In the product, daily job, top 45 only	Summary, why it matches, eligibility notes, deadline, risk, applicant role, two verbatim quotes
Jev and DeepSeek V4 Pro (qualifier)	Offline, reused	Their 1,303 screenings and 283 deep reviews from the qualifier became our reference set for tuning the daily scorer
Claude Code (AI coding assistant)	Development	Wrote and refactored code to our designs, ran the Azure CLI steps, drafted documents we then edited
Organizers' diagram skills (`architecture-security-diagram` , `stakeholder-map`) run with an AI agent	Documentation	Drafted the C4 architecture, STRIDE threat model and stakeholder map, which we checked against the code

Everything sent to Azure OpenAI is public: Grants.gov listing text and the public strategic plan. No staff decisions, names or labels are ever sent to a model.

2. Key prompts

a. The rationale prompt (in the product, `prompts/rationale.txt`). The parts that mattered most:

> *You never invent facts. If the announcement does not state something, write "not stated". Every quote you give must be copied character-for-character from the ANNOUNCEMENT TEXT … a program checks each quote against the source. You do not decide the score or the ranking.*

It is grounded in the listing text plus the one plan objective the scorer matched, with its page number, so the "why it matches" has to argue for a specific objective and can say "this does not really fit".

b. Reusing the qualifier's rules (to the coding assistant). "Map the Grants.gov `fetchOpportunity` response onto the qualifier's CSV-export columns so the tested Layer 1 rules (deadlines, placeholder amounts, cost share, eligibility signals, research mechanisms) run unchanged on live data." This let 71 existing tests keep guarding the daily job.

c. Tuning against evidence instead of guessing (to the coding assistant). "For the grants that are in both today's live pull and the qualifier's reviewed set, compare raw embedding similarity, an embedding

margin against off-topic domains, TF-IDF, and blends. Report AUC against the Jev relevance labels, Spearman against DeepSeek's alignment scores, and precision of the top 50."

d. The diagram skills. We gave the agent the skill file plus a facts sheet of what is built versus planned (e.g. "no Entra ID sign-in yet; secrets are Container Apps secrets, not Key Vault") so the diagrams would not show services we do not run. A second agent then rendered the diagrams and checked every box against the code.

3. Where the AI was wrong, and how we caught it

1. Embeddings ranked unrelated announcements as top matches. On raw similarity, a Navy and Marine Corps research BAA (0.62), a "Countering Violent Extremism" grant (0.61) and a National Park Service cooperative agreement (0.60) scored among the closest matches to the DHHS plan. Long, generic announcements look similar to everything. *How we caught it:* we read the top 25 instead of trusting the numbers. *What we changed:* (a) we also embed ten short descriptions of unrelated fields (defence, arts, physics, agriculture …) and score the **margin**: closer to the DHHS plan than to the nearest unrelated field? (b) we average that with plain TF-IDF word similarity (which is also the fallback if Azure OpenAI is down). Measured on the 1,037 live opportunities the qualifier's pipeline had already reviewed, against whether it judged them on topic for DHHS (`python src/tune_semantic.py`, full table in `docs/tuning_report.md`):

Plan-alignment signal	AUC	Share of top 50 on topic
Raw embedding similarity	0.84	54%
TF-IDF only	0.81	50%
Embedding margin vs off-topic fields	0.91	74%
Hybrid (what we ship)	**0.91**	**76%**

The margin is the real fix; TF-IDF adds a little and keeps the matcher working without AI. The full five-signal score reaches AUC 0.91 against "on topic and DHHS can apply". Its top 50 is slightly less on topic (66–68%) because it also weighs deadlines and award history, which the reference does not judge. These are AI reference labels from the qualifier, used for tuning; the real test is precision@10 on fresh human labels.

A mistake in our own analysis: our first comparison reported lower numbers (0.78 → 0.84, 40% → 68%) because it counted opportunities listed twice in the qualifier data twice and compared capped scores with many ties. Re-running it as a script exposed this; the numbers above are from the corrected script.

2. The model's "verbatim" quotes were not always verbatim. On the first full run, our checker rejected 8 of 90 quotes. Reading each one:

- 5 were our checker being too strict: inside JSON the model swaps `"Domestic"` for `'Domestic'`, and once it quoted the applicant-type list exactly as our prompt displayed it. We fixed the **checker**, not the model.
- The rest were real errors: it **joined two sentences that are not next to each other** into one quote, dropped list numbering ("1. 1 Letter of Commitment" became "1 Letter of Commitment"), returned "N/A" as a quote, and **invented division ids** that are not in our list ("DSCFS" in one run, "DSCFW" twice in the deployed run, apparently blending "DSS" and "DCFW").

How we caught it: every quote is checked against the listing text by code; a failed quote is hidden from staff and logged on the Evaluation page; unknown ids are dropped. An unverified "not eligible" claim is never acted on. In the deployed run on 7 November: **77 of 90 quotes verified word for word, 3 rejected and hidden, the rest left blank** (the model said there was no quote); 6 problems logged in all.

3. Our AI coding assistant introduced a cache bug. Re-running the job showed "0 cached, 1,443 new" embeddings. Three identical re-posted listings were embedded twice but stored once, so the cache's row count no longer matched and it reset itself every run. *Caught by* reading the job log; *fixed* by embedding each unique text once and asserting the cache stays in step.

4. Following a default blindly was slow. The kit's sample deployment command uses capacity 50 (50,000 tokens a minute); the first full embedding pass would have taken about 30 minutes. We raised it to 1,000 for embeddings and 500 for chat (billing is per token, so it costs the same).

4. Guardrails that stay on

- The score is a published formula computed in Python. The model cannot change it; it can only move a record to "not eligible" with a **verified** eligibility quote, or lower confidence one step when it says the match is weak. Both are shown on the record.
- AI runs only on the top 45 records a day (about 30,000 new tokens on a normal day, well under $0.10).
- If Azure OpenAI is down, the job falls back to TF-IDF and skips summaries; the app says so.
- Staff make every decision.